Seguridad del backend del alumno: estado y pendientes

Backend: `aprueba/aprueba_student_web/backend`. Los hallazgos de origen están listados por la empresa en `backend/ENDPOINTS_GAP.md` ("Hallazgos de seguridad preexistentes").

Cerrados en `sec/cierre-hallazgos-acotado`

Hallazgo	Corrección
`POST /auth/social` acepta un idToken falsificable	Fuera de `DEMO_MODE` responde `503`. Dentro, el token demo ya no puede entrar a cuentas con contraseña (`409 ACCOUNT_USES_PASSWORD`).
`JWT_SECRET` cae a `'dev-secret'`	Con `NODE_ENV=production`, el API no arranca si falta, es un valor de ejemplo o tiene menos de 32 caracteres.
`GET /groups/:id/invitations` sin validar membresía	Solo los miembros ven los correos invitados (`403 NOT_GROUP_MEMBER`).
`POST /me/subscription/change` sin validar plan ni pago	Valida plan y ciclo. Sin pagos integrados, solo funciona en `DEMO_MODE`.
`POST /invitations/:token/accept` sin ligar al correo ni vencer	Solo la cuenta del correo invitado acepta (`403 INVITATION_EMAIL_MISMATCH`), y la invitación vence a los 7 días.
Webhook de Stripe sin verificar firma	Deshabilitado siempre (`503 PAYMENTS_DISABLED`).
`cors()` abierto	Solo los orígenes de `CORS_ORIGINS`; en producción, ninguno por defecto.
Sin cabeceras de seguridad	`helmet`.

Todas las correcciones tienen pruebas en `npm run smoke`, sección `[12]`.

`DEMO_MODE`

Mientras no haya usuarios reales, los atajos de prueba quedan detrás de `DEMO_MODE=true` (ver `src/lib/demo.js`):

- activar un plan sin pagar (`/checkout/sessions`, `/confirm`, `/me/subscription/change`);
- login social con el token demo `proveedor:email:nombre`;
- tokens de teléfono `dev:+56912345678`, incluso con `NODE_ENV=production`.

Debe apagarse antes de que entren usuarios reales.

Pendientes

Postergados a propósito: cuestan más, dependen de cuentas o decisiones que todavía no existen, o tocan el frontend. Resolver **antes de tener usuarios reales**.

Pendiente	Qué implica	Por qué se posterga	Depende de
Rate limit robusto (sobre todo en `/auth/*`)	Contadores en un almacén compartido (Redis, o Firestore)	En serverless, un límite en memoria se reinicia en cada instancia.	Elegir el almacén

Pendiente	Qué implica	Por qué se posterga	Depende de
Refresh token en cookie `httpOnly` en web	Cambiar backend y frontend web; CORS con credenciales	Lo exige `Aprueba_API_Backend.docx`. Toca el frontend y el login que se está probando.	—
Rotación y revocación de refresh tokens	Registrar los tokens emitidos en Firestore; invalidar al renovar y en el logout	Hoy duran 30 días y el logout no los revoca. Suma lecturas y escrituras por renovación.	—
Verificar los idToken de Google y Apple en `/auth/social`	Clientes OAuth en Google Cloud y Apple Developer	Apple Developer cuesta US$99 al año.	**Empresa**: ¿login social en el MVP?
Firma del webhook de Stripe	`stripe.webhooks.constructEvent` sobre el cuerpo crudo	Pagos fuera del alcance.	**Empresa**: ¿se implementan pagos?
Teléfono con Firebase Auth real	Números de prueba en Firebase Auth; app Flutter conectada a Firebase	Depende de integrar la app Flutter.	—
Registro desde el frontend web	El frontend web no envía `phoneToken`; el backend lo exige por defecto	Mientras tanto, `PHONE_VERIFICATION_REQUIRED=false` para probar el registro web.	—

Otros hallazgos (consistencia de datos)

No son de seguridad, pero afectan la calidad de los datos. Se detectaron al corregir las respuestas duplicadas (`fix/doble-envio-respuesta`) y quedan pendientes.

Hallazgo	Qué pasa	Impacto	Corrección posible
Repeticiones que no se pueden responder	Si el alumno ya respondió todas las preguntas de la ventana aleatoria, `GET /practice/next` le ofrece una repetida. Como el frontend no envía `sessionId`, `POST /questions/:id/answer` responde `409 ALREADY_ANSWERED` para esa pregunta siempre.	Bajo mientras el banco sea grande (1.365 preguntas). Crece con el uso o con bancos chicos.	Que el cliente envíe un `sessionId` por sesión de práctica, o que `/practice/next` no ofrezca repeticiones que no se pueden responder.
Actualizaciones perdidas en el documento del usuario	La cuota, las medallas y otros contadores se guardan leyendo y reescribiendo el documento `users` completo, sin transacción. Dos peticiones simultáneas del mismo alumno (app y web, o reintentos) pueden pisarse.	Un incremento de cuota o una medalla se pierde en ese caso.	Actualizar los contadores con `FieldValue.increment` o dentro de una transacción de Firestore, como indica `Aprueba_Modelo_Datos_Firestore.docx` (sección 7).